- Edy Lane Martínez Rojas 202510939

- Isabella Casallas Abril 202510253

Project 2

Elementos Esenciales de Lenguajes de Programación

El presente documento es la entrega del Proyecto 1 editada con las correcciones necesarias para implementar Objectilang en Rascal, lo cual corresponde al Proyecto 2. Lo resaltado en amarillo es nuevo o modificado; lo tachado se eliminó. Cada cambio tiene una nota que lo explica: “Corrección P2” para errores o ambigüedades de la gramática original, y “Adaptación P2” para cambios de estructura que no alteran el lenguaje aceptado (debido a que rascal lo tomaba o adaptaba de forma diferente).

Solución del Proyecto 1 utilizada: la de nuestro propio grupo.

Alfabeto

Conjunto de todos símbolos terminales validos en el lenguaje. Es decir, las piezas que aparecen en el código fuente y que ya no se pueden descomponer más dentro de la gramática. Son literalmente escritas.

Σreservadas = {"object", "extends", "method", "val", "var", "new", "override", "if", "then", "else", "for", "in", “String”, “Integer”, “Boolean”, “true”, “false”, “and”, “or”, “neg”}

Corrección P2: and, or y neg son palabras, así que deben ser reservadas. Si no, podrían tomarse como identificadores.

Σoperadores = {"*", "/", "-", "+", "**", "%", "<", ">", "<=", ">=", "<>", "=", "and", "or", "neg", "->", "=>", “<-”}

Σcaracteres = {a, b, ..., z, A, ..., Z}

Σnum = {0, 1, ..., 9}

Σsimbols = {"""", "$", "{", "}", ".", "(", ")", "[", "]", ":", “,”, “ ”, “!”, "${", "\", "//"}

Corrección P2: ${ abre una interpolación como un solo símbolo; \ se usa para escapes dentro de strings (\", \$, \\, \n); // inicia un comentario.

Σotros = {“main”}

Corrección P2: "main" aparece en la regla Metodo pero no estaba en el alfabeto. No es palabra reservada.

Siendo el alfabeto completo: Σ = Σreservadas ∪ Σoperadores ∪ Σcaracteres ∪ Σnum ∪ Σ simbols ∪ Σotros

Conjunto de no-terminales

Símbolos “abstractos” (“variables”) de la gramática que representan conceptos o estructuras del lenguaje (como una “expresión”) en vez del texto literal. Existen para organizar cómo se combinan los terminales.

N = {Programa, Definicion, DefinicionObjeto, DefinicionObjetoMetodo, Metodo, DefinicionGenerico, ConstructorParametros, ParametroConstructor, Parametros, Bloque, Sentencia, DeclaracionVariable, Tipo, Expresion, ExpresionFlecha, ExpresionOr, ExpresionAnd, ExpresionComparacion, OperadorComparacion, ExpresionSuma, ExpresionMult, ExpresionPotencia, ExpresionUnaria, Primario, ElementoBase, Sufijo, Argumentos, NuevaInstancia, CuerpoInstancia, OverrideMetodo, ~~CondicionalIf~~, ~~CicloFor,~~ VariableIteracion, Tupla, Literal, LiteralEntero, LiteralBooleano, LiteralString, CaracterString, ~~Interpolacion~~, Identificador, Letra, Num, Agrupacion, Operacion, Herencia, ParametroGenerico, Varianza, Parametro, Modificador, AnotacionTipo, ParteString, TextoString, Layout, Comentario}

Corrección P2: CondicionalIf y CicloFor pasan a ser alternativas de Expresion, e Interpolacion se integra en ParteString (ver reglas). Los demás no terminales nuevos se explican en sus reglas.

Símbolo distinguido (Axiomas)

Es un solo no-terminal especial, elegido entre los definidos que representa “un programa completo valido en el lenguaje”. Es nuestro punto de partida mediante el cual, aplicando las reglas de producción, se puede derivar cualquier programa valido de nuestro lenguaje de esta tarea.

Símbolo distinguido: Programa

S_0 = Programa

Reglas de producción

Reglas que nos dicen cómo se puede expandir cada no-terminal, ya sea en otros no-terminales, en terminales o en una combinación de ambos. Codifican la sintaxis del lenguaje.

Programa ::= Definicion* Metodo

Definicion ::= DefinicionObjeto | DefinicionObjetoMetodo

Metodo ::= “method” “main” Bloque

DefinicionObjeto ::= “object” Identificador DefinicionGenerico? ConstructorParametros? ~~(“extends” Identificador)?~~ Herencia?

Herencia ::= "extends" Identificador

Adaptación P2: la herencia opcional se separa en su propio no terminal para que el AST tenga un constructor claro. El lenguaje aceptado no cambia.

ParametroConstructor ::= “val” Identificador “:” Tipo

ConstructorParametros ::= “(“ ParametroConstructor (“,” ParametroConstructor)* “)”

DefinicionObjetoMetodo ::= Identificador “.” Identificador “=” “method” “(” Parametros “)” “:” Tipo Bloque

~~Parametros ::= (Identificador “:” Tipo (“,” Identificador “:” Tipo)*)?~~

Parametros ::= (Parametro "," Parametro)*)?

Parametro ::= Identificador ":" Tipo

Adaptación P2: se nombra cada parámetro como no terminal. El lenguaje no cambia.

Identificador :: = Letra (Letra | Num)*

~~DefinicionGenerico ::= “[” (“+” | “-”)? Identificador (“,” (“+” | “-”)? Identificador)* “]”~~

DefinicionGenerico ::= "[" ParametroGenerico ("," ParametroGenerico)* "]"

ParametroGenerico ::= Varianza? Identificador

Varianza ::= "+" | "-"

Adaptación P2: el grupo anónimo ("+" | "-")? se reemplaza por reglas con nombre. El lenguaje no cambia.

Bloque ::= “{” Sentencia* “}”

Corrección P2: sin separador entre sentencias, { x -1 } podía leerse como una resta o como dos sentencias (x y -1), y { print (1) } como una llamada o como print seguido de (1). Se agregó la restricción de que una sentencia no puede ir seguida de otra que empiece por (o -; en ese caso se lee como continuación de la expresión. Esta restricción no se puede escribir en EBNF; en Rascal se expresa con () !>> "(" !>> "-".

Sentencia ::= DeclaracionVariable | Expresion ~~| Bloque~~

Corrección P2: un bloque ahora es una expresión (ver ExpresionUnaria). Dejarlo también aquí daba dos derivaciones para el mismo bloque.

~~DeclaracionVariable ::= (“val” | “var”) Identificador (“:” Tipo)? “=” Expresion~~

DeclaracionVariable ::= Modificador Identificador AnotacionTipo? "=" Expresion

Modificador ::= "val" | "var"

AnotacionTipo ::= ":" Tipo

Adaptación P2: los grupos ("val" | "var") y (":" Tipo)? se separan en reglas con nombre. El lenguaje no cambia.

~~Expresion ::= ExpresionFlecha~~

Expresion ::= "if" Expresion "then" Expresion ("else" Expresion)? | "for" "(" VariableIteracion "<-" Expresion ")" Expresion | Operacion

Operacion ::= ExpresionFlecha

Corrección P2: en P1, if y for estaban dentro de ElementoBase, así que podían quedar dentro de una operación, y había frases con dos lecturas. Por ejemplo, if a then b else c + 1 podía ser (if a then b else c) + 1 o if a then b else (c + 1). Ahora if y for están en un nivel aparte: los operadores solo combinan Operacion, así que un if/for dentro de una operación debe ir entre paréntesis: x + (if a then b else c). Sus ramas pasan de Sentencia a Expresion. Además, se resuelve el else colgante: en if a then if b then c else d, el else pertenece siempre al if más cercano (un if sin else no puede quedar seguido de else; en Rascal, () !>> "else").

ExpresionFlecha ::= ExpresionOr ((“->” | “=>”) ExpresionOr)*

ExpresionOr ::= ExpresionAnd (“or” ExpresionAnd)*

ExpresionAnd ::= ExpresionComparacion (“and” ExpresionComparacion)*

ExpresionComparacion ::= ExpresionSuma (OperadorComparacion ExpresionSuma)?

ExpresionSuma ::= ExpresionMult ((“+” | “-”) ExpresionMult)*

ExpresionMult ::= ExpresionPotencia ((“*” | “/” | “%”) ExpresionPotencia)*

ExpresionPotencia ::= ExpresionUnaria (“**” ExpresionPotencia)?

ExpresionUnaria ::= “neg” ExpresionUnaria | “-” ExpresionUnaria | Primario | Bloque

Corrección P2: así un bloque { … } puede ser rama de un if o cuerpo de un for (por ejemplo, if c then { … } else { … }). En Rascal, la cascada ExpresionFlecha … ExpresionUnaria se implementa con un solo no terminal Operacion, con prioridades y asociatividad que reproducen exactamente esta precedencia.

Primario ::= ElementoBase ~~Argumentos?~~ Sufijo*

ElementoBase ::= Identificador | Literal | NuevaInstancia ~~| CondicionalIf | CicloFor~~ | Tupla | Agrupacion | Identificador Argumentos

Corrección P2: con Argumentos? en Primario, new Cat(1) tenía dos derivaciones (los argumentos podían ser del new o del Primario), y además permitía cosas como 1(2). Ahora los argumentos solo van pegados a un identificador (print(x)), a un método (c.talk()) o a un new. CondicionalIf y CicloFor se movieron a Expresion.Sufijo ::= “.” Identificador Argumentos?

Literal ::= LiteralEntero | LiteralBooleano | LiteralString

LiteralEntero ::= Num+

LiteralBooleano ::= “true” | “false”

~~LiteralString ::= “‘” (CaracterString | Interpolacion)* “’”~~

~~Interpolacion ::= “$” “{“ Expresion “}” | “$” Identificador~~

~~CaracterString ::= Letra | Num | “ “ | “.” | “,” | “:” | “!”~~

LiteralString ::= "" ParteString* ""

ParteString ::= TextoString | "$" Identificador | "${" Expresion "}"

TextoString ::= CaracterString+

CaracterString ::= (cualquier carácter excepto "", "$", "\" y salto de línea) | "\" ("" | "$" | "\" | "n")

Corrección P2: (1) CaracterString solo admitía letras sin tilde, dígitos, espacio y . , : !, así que strings como "¿Cómo estás?" o "100%" eran inválidos. Ahora se acepta cualquier carácter salvo los especiales, que se escriben con escape. (2) Dentro de un string los espacios son parte del texto y no se descartan; solo se permiten espacios ignorables dentro de ${ … }. En Rascal, LiteralString es léxico por esta razón.

~~CondicionalIf ::= “if” Expresion “then” Sentencia (“else” Sentencia)?~~

~~CicloFor ::= “for” “(“ VariableIteracion “<-” Expresion “)” Sentencia~~

Corrección P2: ahora son alternativas de Expresion (ver arriba).

VariableIteraci~~ó~~on ::= Identificador | “(“ Identificador “,” Identificador (“,” Identificador)* “)”

Corrección P2: el nombre tenía tilde aquí, pero no en N.

Agrupacion ::= “(” Expresion “)”

Tupla ::= “(“Expresion “,” Expresion (“,” Expresion)* “)”

NuevaInstancia ::= “new” Tipo Argumentos? CuerpoInstancia?

CuerpoInstancia ::= “{“ OverrideMetodo+ “}”

OverrideMetodo ::= “override” Identificador “(“ Parametros? “)” “:” Tipo “=” Bloque

Argumentos ::= “(“(Expresion (“,” Expresion)*)? “)”

OperadorComparacion ::= "<" | ">" | "<=" | ">=" | "<>" | "=" | “in”

Letra ::= (“A” | ... | “Z” | “a” | ... | “z”)

Num ::= (0| ... | 9)

~~Tipo ::= (“String” | “Integer” | “Boolean” | Identificador) ("[" Tipo ("," Tipo)* "]")?~~

Tipo ::= "String" | "Integer" | "Boolean" | Identificador | Identificador "[" Tipo ("," Tipo)* "]"

Corrección P2: la regla original permitía String[Integer] o Boolean[A]. Solo un tipo con nombre (como List) puede llevar argumentos genéricos.

Layout ::= (" " | tabulación | salto de línea | Comentario)*

Comentario ::= "//" (cualquier carácter excepto salto de línea)*

Adaptación P2: P1 no definía espacios ni comentarios. Layout puede aparecer entre cualquier par de símbolos de una regla (excepto dentro de identificadores, números y strings) y se ignora.

Nota: La regla Identificador genera sintácticamente cualquier cadena que empiece con una letra, seguida de letras o dígitos. Sin embargo, ninguna cadena que coincida directamente con una palabra reservada del alfabeto de reservadas puede clasificarse como Identificador. Este tipo de restricción no puede expresarse en EBNF formalmente.

Corrección P2: en Rascal esta restricción sí se puede expresar: Id = ([a-zA-Z][a-zA-Z0-9]* !>> [a-zA-Z0-9]) \ Reservada. Además, un identificador siempre toma la palabra completa, y una palabra reservada no puede ir pegada a una letra o dígito. En P1, negative podía leerse como el identificador negative o como neg aplicado a ative (igual con valores y newton). Ahora siempre es un identificador.

Derivaciones

Derivación 1: object Dog extends Animal

~~DefinicionObjeto --> "object" Identificador DefinicionGenerico? ConstructorParametros? ("extends" Identificador)?~~

~~--> "object" Identificador ("extends" Identificador)~~

~~[se omiten DefinicionGenerico y ConstructorParametros]~~

~~--> "object" Dog ("extends" Identificador)~~

~~[Identificador: Dog]~~

~~--> "object" Dog "extends" Animal~~

~~[Identificador: Animal]~~

~~**object Dog extends Animal**~~

DefinicionObjeto --> "object" Identificador DefinicionGenerico? ConstructorParametros? Herencia?

--> "object" Identificador Herencia [se omiten DefinicionGenerico y ConstructorParametros]

--> "object" Dog Herencia [Identificador: Dog]

--> "object" Dog "extends" Identificador [Herencia]

--> "object" Dog "extends" Animal [Identificador: Animal]

Derivación 2: object List[+A](val item: A)

DefinicionObjeto --> "object" Identificador DefinicionGenerico? ConstructorParametros? Herencia?

--> "object" Identificador DefinicionGenerico ConstructorParametros

[se omite "extends"]

--> "object" List DefinicionGenerico ConstructorParametros

[Identificador: List]

--> "object" List DefinicionGenerico ConstructorParametros [Identificador: List; se omite Herencia]

--> "object" List "[" ParametroGenerico "]" ConstructorParametros

--> "object" List "[" Varianza Identificador "]" ConstructorParametros

--> "object" List "[" "+" A "]" ConstructorParametros [Varianza: "+"; Identificador: A]

~~--> "object" List "[" ("+"|"-")? Identificador "]" ConstructorParametros~~

~~--> "object" List "[" "+" Identificador "]" ConstructorParametros~~

~~[se elige "+"]~~

~~--> "object" List "[" "+" A "]" ConstructorParametros~~

~~[Identificador: A]~~

~~--> "object" List "[" "+" A "]" "(" ParametroConstructor ")"~~

~~--> "object" List "[" "+" A "]" "(" "val" Identificador ":" Tipo ")"~~

~~--> "object" List "[" "+" A "]" "(" "val" item ":" Tipo ")"~~

~~[Identificador: item]~~

~~--> "object" List "[" "+" A "]" "(" "val" item ":" A ")"~~

~~[Tipo: Identificador : A]~~

~~**object List[+A](val item: A)**~~

Derivación 3: Cat.talk = method(): String { print("Cats meows") }

Definicion

--> DefinicionObjetoMetodo

--> Identificador "." Identificador "=" "method" "(" Parametros ")" ":" Tipo Bloque

--> Cat "." Identificador "=" "method" "(" Parametros ")" ":" Tipo Bloque [Identificador: Cat]

--> Cat . talk "=" "method" "(" Parametros ")" ":" Tipo Bloque [Identificador: talk]

--> Cat.talk = method(Parametros) : Tipo Bloque

--> Cat.talk = method( ) : Tipo Bloque [Parametros: vacío, sin argumentos]

--> Cat.talk = method(): String Bloque [Tipo: "String"]

--> Cat.talk = method(): String "{" Sentencia* "}"

--> Cat.talk = method(): String { Sentencia } [una sola Sentencia]

--> Cat.talk = method(): String { Expresion } [Sentencia: Expresion]

--> Cat.talk = method(): String { Operacion } [Expresion: Operacion]

--> ExpresionFlecha

--> ExpresionOr

--> ExpresionAnd

--> ExpresionComparacion

--> ExpresionSuma

--> ExpresionMult

--> ExpresionPotencia

--> ExpresionUnaria

--> Primario [cae por toda la jerarquía]

--> Cat.talk = method(): String { Primario }

--> Cat.talk = method(): String { ElementoBase Sufijo* }

--> Cat.talk = method(): String { Identificador Argumentos } [ElementoBase: Identificador Argumentos; sin Sufijo]

--> Cat.talk = method(): String { print Argumentos } [Identificador: print]

--> Cat.talk = method(): String { print "(" Expresion ")" } [Argumentos con una sola Expresion]

--> Cat.talk = method(): String { print(Operacion) } [Expresion: Operacion]

--> ... --> Primario --> ElementoBase Sufijo* --> Literal

--> LiteralString [cae de nuevo por la jerarquía; sin Sufijo]

--> Cat.talk = method(): String { print("" ParteString* "") }

--> Cat.talk = method(): String { print("" TextoString "") } [una sola ParteString: TextoString]

--> Cat.talk = method(): String { print("" CaracterString+ "") }

--> Cat.talk = method(): String { print("Cats meows") } [CaracterString: C, a, t, s, espacio, m, e, o, w, s]

Cat.talk = method(): String { print("Cats meows") }

Gracias:D